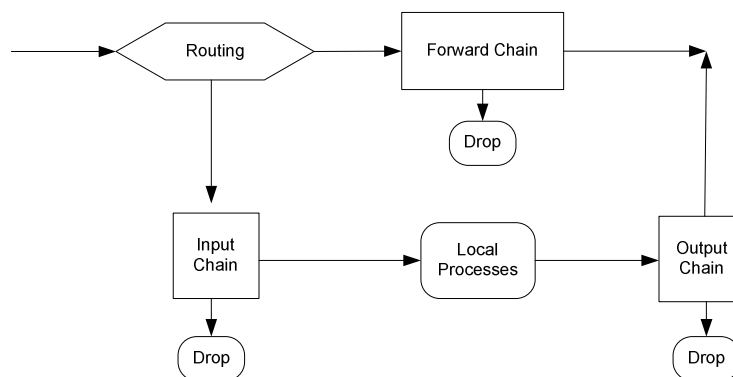


บทที่ 5

การพัฒนาไฟร์วอลล์โดยใช้ไอพีเทเบิล

ไอพีเทเบิล (iptables) และ เนตฟิลเตอร์ (netfilter) เป็นเฟรมเวิร์กที่อยู่ใน เคอร์เนล(kernel) ลินุกซ์ 2.4 ซึ่งจะสามารถทำการ แพ็กเกจฟิลเตอร์ริง (Package filtering) การทำเน็ตเวิร์กแอดเดรสทรานสเลชัน (Network Address Translation NAT) การตรวจสอบสเตทของการติดต่อ (State Inspection) และการทำกั้นกรองแพ็กเกจในแบบอื่นๆ โดยจะมีการทำงานที่เข้าถึง เน็ตเวิร์คสแตค ภายในเคอร์เนล ลินุกซ์ 2.4(Linux 2.4.x kernel's network stack) ซึ่งจะทำให้เคอร์เนลอนุญาตให้ทำ register callback functions ทุกครั้งที่แพ็กเกจผ่านเข้ามา โดยจะมีการทำงานโดยการอิงชุดของกฎ(Rulesets) เข้ากับหมายเลขไอพี นอกจากนี้ยังทำการรวมเอาส่วนของเนต (NAT) เข้าไปกับเฟรมเวิร์คของไอพีเทเบิลได้ด้วย

การทำงานของไอพีเทเบิล นั้นจะประกอบด้วยอินพุทเชน(INPUT chain) เอาท์พุทเชน (OUTPUT chain) และ ฟอว์เวิร์ดเชน (FORWARD chain) โดยแพ็กเกจขาเข้า (incoming Packet) จะผ่านเราต์ติ้งฟังก์ชัน (routing function) เพื่อวิเคราะห์ว่า แพ็กเกจ นั้นๆ จะผ่านเข้าไปตรวจสอบกับฟอว์เวิร์ดเชนหรืออินพุทเชน (Input chain) ดังรูป 5-1



รูปที่ 5-1 Netfilter packet traversal (จาก “Linux 2.4 Packet Filtering HOWTO, by Rusty Russel, v.1.0.1)

หากเป็นแพ็กเกจที่มาจากโลคอลแอดเดรส (Local Address) และผ่านอินพุทเชน แพ็กเกจนั้นก็จะถูกส่งไปยังโลคอลแอดเดรสที่กำหนดในแพ็กเกจ แต่หากเป็นแพ็กเกจจากภายนอก ที่ต้องการผ่านเข้ามาในไฟร์วอลล์ ก็จะถูกเทียบกับฟอว์เวิร์ดเชน หากผ่านตามกฎแพ็กเกจนั้น ก็จะถูกส่งไปยังอินเทอร์เน็ตที่เหมาะสมตามที่กำหนดไว้

สำหรับแพ็กเกจขาออก (Outgoing packets) ที่เป็นโลคอลแอดเดรส และผ่านเอาท์พุทเชน แพ็กเกจ นั้นก็จะถูกส่งไปยังอินเทอร์เน็ตที่เหมาะสมตามที่กำหนดไว้

5.1 ส่วนประกอบของ ไอพีเทเบิล

5.1.1 ฟิวเตอร์เทเบิล (Filter table)

เป็น ส่วนที่จะดูแลการกรองแพ็กเกจต่างๆ ตามกฎที่กำหนดไว้ซึ่งจะเป็นส่วนหลักในการทำงานซึ่งจะมีความสามารถในการทำงานที่พิเศษดังนี้

- ตรวจสอบสถานะของการติดต่อซึ่งสามารถที่จะเข้าถึง ทีซีพีสเตตแฟล็ก (TCP state flags) และ ทีซีพีออปชั่นฟิลด์ (TCP option field) ได้
- กำหนดพอร์ตในการทำงานแบบหลายพอร์ต
- ทำการตรวจสอบการทำงานของอีเทอร์เน็ตแอดเดรสของอุปกรณ์ (MAC Address)
- เข้าถึงฟิลด์ข้อมูลรูปแบบของการบริการในทีซีพี (TOS) โดยทำงานร่วมกับ แมนเกิลเทเบิล
- ทำการควบคุมอัตราของข้อมูล
- ทำการใส่ค่าเข้าไปใน มาร์กฟิลด์ (packet's mark field)
- ทำการตรวจสอบการใช้งานของผู้ใช้โดยการอิงไอดีของผู้ใช้ (User ID), ไอดีของกลุ่มของผู้ใช้ (Group ID) หรือ โพรเซสไอดี (Process ID)

5.1.2 แนทเทเบิล (NAT Table)

เป็นส่วนที่ดูแลการทำแนท เช่น การทำ SNAT (Source NAT), DNAT (Destination NAT), MASQUERADE และการทำ REDIRECT โดยการทำงานนั้นจะแบ่งออกเป็น 3 แบบคือ

- การทำแนทแบบเดิม หรือ การทำแนทแบบทางเดียวออกภายนอก (Traditional, unidirectional outbound NAT) จะเป็นการทำงานที่ใช้งานบน เครือข่ายที่มีการใช้ไฟร์วอลล์ โดยจะมีสองแบบ คือ
 - แนทแบบธรรมดา จะทำการทำแผนผังเทียบ แอดเดรสที่เป็นไฟร์วอลล์ในเครือข่ายภายใน ออกเป็นแอดเดรสที่เป็นเครือข่ายสาธารณะ
 - แนทที่ทำการเปลี่ยนพอร์ตของเครือข่าย (Network Address Port Translation) จะทำการเปลี่ยนจากไฟร์วอลล์แอดเดรสภายในเป็นแอดเดรสสาธารณะเพียงแอดเดรสเดียว
- การทำแนทแบบสองทาง (Bidirectional NAT) จะทำการเปลี่ยนแอดเดรสทั้งการส่งข้อมูลออกจากระบบ และการรับข้อมูลจากภายนอกเข้ามาใช้งานเครื่องที่อยู่ภายใน ซึ่งสามารถทำงานได้ทั้งไอพีเวอร์ชันสี่ (IPv4) และ ไอพีเวอร์ชันหก (IPv6) ด้วย
- การทำแนทสองครั้ง (Twice NAT) จะเป็นการทำการแนทเมื่อมีการเปลี่ยนแปลงแอดเดรสสาธารณะ เนื่องจากหากมีการกำหนดแอดเดรสให้กับ

เครื่องที่เป็นแอดเดรสที่จะทำการเนทไปนั้น จะต้องมีการเปลี่ยนแปลงการเนทใหม่ แต่การทำเนทสองครั้งนั้นจะทำการติดตามการเปลี่ยนแปลงแอดเดรสสาธารณะ และทำการปรับเปลี่ยนการเนทให้เหมาะสมกับแอดเดรสที่ได้รับใหม่ทำให้ผู้บริหารระบบไม่จำเป็นต้องติดตามการเปลี่ยนแปลงและทำการเปลี่ยนแปลงค่าที่ตั้งไว้ใหม่

5.1.3 แมนเกลเทเบิล (Mangle table)

เป็นส่วนที่จะดูแลแพ็กเกจ แบบพิเศษกว่าฟิลเตอร์เทเบิล โดยจะทำการเปลี่ยนแปลงที่ TOS ฟิลด์ เพื่อการตัดสินใจในการทำโลคอลเรทติ้งหรือฟอร์เวิร์ดดิ้งซึ่งจะแบ่งเป็น

- พรีเรทติ้ง (PREROUTING) จะทำการเปลี่ยนแปลงแพ็กเกจขาเข้า (Incoming Packet) ก่อนที่จะทำการเรทติ้ง
- เอาต์พุท จะทำการเปลี่ยนแปลงแพ็กเกจ ที่จะส่งออกไปจากโลคอลแพ็กเกจ

5.2 การใช้งานไอพีเทเบิล

ในแต่ละเทเบิลนั้นจะมีการสร้างเชนที่มากับเทเบิลอยู่ดังนี้

- ฟิลเตอร์เทเบิล จะประกอบด้วยเชนหลักอยู่ 3 เชน คือ
 - อินพุตเชน
 - เอาต์พุตเชน
 - ฟอร์เวิร์ดเชน
- เนทเทเบิล
 - พรีเรทติ้งเชน (สำหรับการทำ DNAT กับ REDIRECT)
 - เอาต์พุตเชน (สำหรับการทำ DNAT กับ REDIRECT)
 - โฟสเรทติ้งเชน (สำหรับการทำ SNAT กับ REDIRECT)
- แมนเกลเทเบิล
 - พรีเรทติ้งเชน (สำหรับการทำการเรทติ้งแพ็กเกจ)
 - เอาต์พุตเชน (สำหรับการสร้างแพ็กเกจใหม่ขึ้นภายใน)

การใช้งานไอพีเทเบิล

- คำสั่งที่ใช้กับเชน

การสร้างเชนใหม่

```
iptables -N name_of_chain
```

การลบเชนที่สร้างขึ้นมาเอง

```
iptables -X name_of_chain*
```

*ถ้า name_of_chain ไม่มี จะทำการลบ เชนที่สร้างขึ้นเองทั้งหมด
การเปลี่ยนนโยบายของเชนที่อยู่ภายใน

iptables -P name_of_chain name_of_policy*

*DROP หรือ ACCEPT

การแสดงกฎทั้งหมดที่อยู่ในเชน

iptables -L name_of_chain

การลบกฎในเชน

iptables -F name_of_chain*

*ถ้า name_of_chain ไม่มี จะทำการลบกฎทั้งหมดในเชน

การตั้งตัวนับแพ็กเก็ต และตัวนับข้อมูลของกฎในเชน

iptables -Z name_of_chain

○ คำสั่งที่เกี่ยวข้องกับกฎ

การเปลี่ยนเทเบิล (-t table_name). หาก table_name ไม่มีจะหมายถึงฟิลเตอร์เทเบิล

การกระทำกับเชน จะมี การเพิ่มไปที่กฎแรก (-A), การลบ (-D), การแทนที่ (-R) และการเพิ่มที่กฎสุดท้าย (-I) จากนั้นตามด้วยชื่อเชนในเทเบิลนั้นๆ

การตรวจสอบต่างๆ

-p (โปรโตคอล: TCP, UDP, ICMP หรืออื่นๆ)

-s (แอดเดรสต้นทาง)

-d (แอดเดรสปลายทาง)

-i (อินเทอร์เฟซที่เข้า)

-o (อินเทอร์เฟซออก)

--fragment (ตรวจสอบการแฟรกเมนต์)

--dport (พอร์ตปลายทาง)

--sport (พอร์ตต้นทาง)

--port (พอร์ตทั้งต้นทางและปลายทางที่เท่ากับที่กำหนด)

--mark (ค่าของ nfmak)

--tcp-flags (ตรวจสอบแฟรกต่างๆ)

--syn (การทำ SYNC Attack)

--tcp-option (ค่าของทีซีพีออปชั่น)

--state (NEW, ESTABLISHED, RELATED, INVALID)

--mac-source (source MAC address)

--tos (ค่า TOS)

--ttl (ค่า ttl)

○ การควบคุมอัตราของข้อมูล

--limit (อัตราของข้อมูล)

--limit-burst (ค่ามากที่สุดในการ burst ข้อมูล)

○ การควบคุมอัตราของข้อมูลตามไอพี

--limit-above จำกัดจำนวนการติดต่อของ ทีซีพี กับไอพีที่กำหนดไว้

○ การควบคุมตามผู้ส่ง

--uid-owner userid (ควบคุมตามไอดีของผู้ใช้)

--gid-owner groupid (ควบคุมตามไอดีของกลุ่มผู้ใช้)

--pid-owner processid (ควบคุมตามไอดีของโปรเซส)

--sid-owner sessionid (ควบคุมตามไอดีของเซสชัน)

○ เป้าหมายในการทำกับแพ็กเก็ตที่ตรงกับกฎนั้นๆ:

ACCEPT – รับแพ็กเก็ต

DROP – ทิ้งแพ็กเก็ตนั้น

REJECT – ทิ้งแพ็กเก็ตนั้นและแจ้งไปยังผู้ส่ง

LOG – บันทึกลงล็อก แล้วทำงานต่อ

MIRROR – สลับแอดเดรสผู้รับและผู้ส่งและทำงานต่อ

QUEUE – ทำการส่งเข้าไปใน ยูสเซอร์สเปซโปรเซสคิว(userspace process queue)

RETURN – กลับไปยังเงื่อนไขทำการเรียกมา

ผลลัพธ์ที่ได้จากการใช้คำสั่ง 'iptables -h'

iptables v1.2.2

```
Usage: iptables -[ADC] chain rule-specification [options]
       iptables -[RI] chain rulenum rule-specification [options]
       iptables -D chain rulenum [options]
       iptables -[LFZ] [chain] [options]
       iptables -[NX] chain
       iptables -E old-chain-name new-chain-name
```

```
iptables -P chain target [options]
iptables -h (print this help information)
```

Commands:

Either long or short options are allowed.

```
--append -A chain          Append to chain
--delete -D chain          Delete matching rule from chain
--delete -D chain rulenum  Delete rule rulenum (1 = first) from
chain
--insert -I chain [rulenum] Insert in chain as rulenum (default
1=first)
--replace -R chain rulenum Replace rule rulenum (1 = first) in
chain
--list -L [chain]         List the rules in a chain or all
chains
--flush -F [chain]       Delete all rules in chain or all
chains
--zero -Z [chain]        Zero counters in chain or all chains
--check -C chain         Test this packet on chain
--new -N chain           Create a new user-defined chain
--delete-chain           Delete a user-defined chain
-X [chain]
--policy -P chain target Change policy on chain to target
--rename-chain           Change chain name, (moving any
-E old-chain new-chain
references)
Options:
--proto -p [!] proto     protocol: by number or name, eg.
'tcp'
--source -s [!] address[/mask]
                        source specification
--destination -d [!] address[/mask]
                        destination specification
--in-interface -i [!] input name[+]
                        network interface name ([+] for
wildcard)
--jump -j target         target for rule (may load target
extension)
--match -m match         extended match (may load extension)
--numeric -n            numeric output of addresses and ports
--out-interface -o [!] output name[+]
                        network interface name ([+] for
wildcard)
--table -t table        table to manipulate (default:
'filter')
--verbose -v           verbose mode
--line-numbers         print line numbers when listing
--exact -x            expand numbers (display exact values)
[!] --fragment -f      match second or further fragments
only
--modprobe=            try to insert modules using this command
--set-counters PKTS BYTES set the counter during insert/append
[!] --version -V       print package version.
```

5.3 การพัฒนาไฟร์วอลล์โดยใช้ไอพีเทเบิล

เนื่องจากที่ ไอพีเทเบิล ทำงานในลักษณะอิงแอดเดรส (Address-Based) แต่ต้องการพัฒนาให้อิงผู้ใช้ (User-based) แนวคิดของการพัฒนาคือการให้อิงแอดเดรส ในขณะเวลาหนึ่งที่ผู้ใช้ คนนั้นล็อกอิน (login) อยู่ โดยต้องมีส่วนที่คอยตรวจสอบว่ามี ผู้ใช้ ทำงานอยู่ที่แอดเดรสใด และมีส่วนที่คอยรับผลจาก โปรแกรมข้างต้น นำมาทำการสร้างกฎที่เหมาะสม คือ เป็นกฎสำหรับผู้ใช้ที่ล็อกอิน อยู่ที่แอดเดรสนั้นๆ แล้วทำการเพิ่ม เข้าไปและคอยตรวจสอบว่า ผู้ใช้ที่อยู่แอดเดรสนั้นเปลี่ยนแปลงหรือไม่ และคอยทำการปรับเปลี่ยนกฎให้เหมาะสมกับผู้ใช้ปัจจุบันที่ล็อกอิน อยู่ที่แอดเดรสนั้นๆ โดยในโครงการนี้ ส่วนที่คอยติดตามรับผลที่ได้จากการตรวจสอบของโปรแกรมที่อยู่ในส่วนของวินโดวส์2000 ที่จะคอยติดตามการเปลี่ยนแปลงของ ผู้ใช้ ที่ทำการล็อกอินผ่านแอคทีฟไดเรกทอรี (Active Directory) แล้วทำการส่งข้อมูลในเกี่ยวกับการเปลี่ยนแปลง ผู้ใช้ มายังเดมอน (Daemon) ที่อยู่บน ลินุกซ์ ผ่าน โปรโตคอลที่กำหนดขึ้นมาเอง เมื่อ โปรแกรมที่อยู่บน ลินุกซ์ ได้รับข้อมูลของการเปลี่ยนแปลง ซึ่งจะประกอบไปด้วย ชื่อผู้ใช้ กับ หมายเลขไอพี (IP Address) ก็จะนำชื่อผู้ใช้ ไปค้นหากฎที่ทำการเก็บอยู่ใน แฟ้มข้อมูล แล้วนำมาใส่ใน ส่วนของหมายเลขไอพี แล้วทำการ เพิ่ม หรือ ลบ กฎ ลงไปในไอพีเทเบิล ซึ่งเดมอน ที่อยู่บนลินุกซ์ จะทำการคอยรับที่พอร์ต ที่ทำการตั้งไว้ตลอดเวลา เมื่อมีข้อมูลส่งมาโปรแกรมจะรับข้อมูลมาทำงานทันที และในการทำงานจะมีการทำงานอยู่ สองแบบ คือ ทำงานเป็นเบื้องหลัง(background mode) และเมื่อมีการกระทำใดๆ ก็จะมีการ เก็บข้อมูลลงในล็อกไฟล์ (log file) ซึ่งข้อมูลที่จะเก็บจะประกอบไปด้วย

- หมายเลขไอพี
- เวลา
- ผู้ใช้ ที่ล็อกอินอยู่ที่หมายเลขไอพีนั้นๆ
- กฎที่ตั้งไว้สำหรับผู้ใช้ นั้นๆ
- การกระทำที่ทำกับไอพีเทเบิล (เพิ่ม หรือ ลบ กฎ)

ส่วนอีกแบบหนึ่งคือการทำงานเป็น โพรเซส (Process) ทั่วไป และมีการแสดงผลที่ได้ออกมายังคอนโซล (Console) เหมือนโปรแกรมทั่วไป

ในส่วนของการติดต่อกับไอพีเทเบิลนั้น เนื่องจากทางผู้พัฒนาโปรแกรมไม่ได้ทำการสร้างเอพีไอ (API) เพื่อให้ โปรแกรมเมอร์ ได้ติดต่อกับ ตัวโปรแกรม ดังนั้นจึงจำเป็นต้องใช้การฟอร์คโพรเซส (fork process) มาเพื่อทำการเอ็กซ์คิว (Execute) โปรแกรม ไอพีเทเบิล ในโหมดที่ทำการ เพิ่มหรือลบกฎต่าง เมื่อ ไอพีเทเบิล ทำงานเสร็จเรียบร้อยแล้วก็จะทำการส่งค่ากลับออกมาโพรเซส ที่ฟอร์คขึ้นมาก็จะตายไป แล้ว โปรแกรมก็จะกลับมาทำงานต่อตามปกติดังตัวอย่างโปรแกรม

```
#include <stdio.h>
```

```
#include <sys/types.h>
```

```
void parse(char *line, char **argv)
```

```
{
```

```

        while (*line != '\0') {    /* if not the end of line ..... */
while (*line == ' ' || *line == '\t' || *line == '\n')
    *line++ = '\0';    /* replace white spaces with 0 */
*argv++ = line;    /* save the argument position */
while (*line != '\0' && *line != ' ' &&
    *line != '\t' && *line != '\n')
    line++;    /* skip the argument until ... */
        }
        *argv = '\0';    /* mark the end of argument list */
    }
}

void execute(char **argv)
{
    pid_t pid;
    int status;

    if ((pid = fork()) < 0) {    /* fork a child process */
        printf("*** ERROR: forking child process failed\n");
        exit(1);
    }

    else if (pid == 0) {    /* for the child process: */
        if (execvp(*argv, argv) < 0) {    /* execute the command */
            printf("*** ERROR: exec failed\n");
            exit(1);
        }
    }

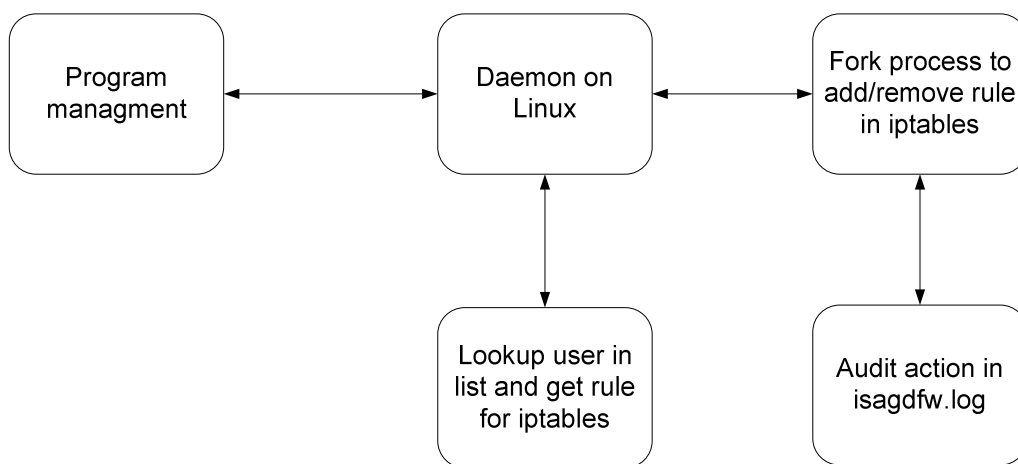
    else {    /* for the parent: */
        while (wait(&status) != pid);    /* wait for completion */
    }
}

```

รูปที่ 5-2 source code file exec.c

จากรูปที่ 5-2 นั้นไฟล์ exec.c นั้นจะมีฟังก์ชัน อยู่ 2 ฟังก์ชัน คือ parse() และ execute() โดยการทำงานนั้นรู้ที่ execvp() นั้นจะต้องการอาร์กิวเมนต์ (argument) อยู่ สองตัว โดยที่ตัวแรกจะเป็นชื่อของ

โปรแกรม ที่ต้องการที่จะทำการเอ็กซ์คิว ส่วนอาร์กิวเมนต์ ที่สองจะเป็นอาร์เรย์ (Array) ของตัวชี้ (Pointer) ไปยังตัวอักษร (character) โดยจะเป็นอาร์กิวเมนต์ ที่จะใส่เข้าไปในโปรแกรมที่ต้องการเอ็กซ์คิว ดังนั้นจึงมีฟังก์ชัน parse() ขึ้นมาเพื่อทำการตัดสตริง (String) ที่ได้จากการอ่านไฟล์ ที่เก็บ กฎเอาไว้ มาใส่ไว้ในอาร์เรย์ เพื่อเตรียมให้ฟังก์ชัน execvp() ทำการใช้งาน โดยเมื่อทำการตัดสตริงออกมาเป็นอาร์เรย์ แล้วก็ส่งค่าไปยังฟังก์ชัน execute() ฟังก์ชันก็จะทำการโหลดไพรเซสขึ้นมา แล้วทำการเอ็กซ์คิวโปรแกรมแล้วรอจนกว่าไพรเซสที่ทำการเอ็กซ์คิวโปรแกรม เสร็จเรียบร้อยแล้ว ก็จะทำการส่งค่าฟังก์ชันกลับ (return function) เพื่อกลับไปทำงานในโปรแกรมหลัก (main program) ต่อไป



รูปที่ 5-3 รูปแสดงการทำงานของ โปรแกรมฝังลินุกซ์

จากรูปที่ 5-3 ซึ่งจะแสดงการทำงานของโปรแกรมที่อยู่บนฝังลินุกซ์ โดยการทำงานนั้นจะเป็นลักษณะของ เดมอน (Daemon process) คือจะทำการรอรับการติดต่อจากโปรแกรมที่อยู่บนฝัง วินโดวส์ 2000 ซึ่งจะทำการตรวจสอบการเปลี่ยนแปลงของผู้ใช้งานที่ทำการล็อกอินอยู่ที่ไคลเอนท์ แล้วจะทำการส่งข้อมูลมายังส่วนที่อยู่บนลินุกซ์เมื่อมีการเปลี่ยนแปลง โดยจะส่งมาในรูปแบบดังนี้

<User>:<ip address><command>

โดยในการส่งข้อมูลนั้นคำสั่ง จะเป็นการสั่งให้ทำการ เพิ่ม หรือ ลบกฎออกจากไอพีเทเบิลนั่นเอง โดยเมื่อโปรแกรมที่อยู่บนฝั่งลินุกซ์ได้รับข้อความจากฝั่งวินโดวส์ 2000 นั้น ก็จะทำการแยกข้อมูลที่ได้รับมา ออกเป็นสามส่วน แล้วทำการนำเอาชื่อผู้ใช้ไปค้นหาจากเพิ่มข้อมูลที่เก็บกฎของผู้ใช้ไว้ จากนั้นก็ทำการสร้างโพสเซสใหม่ขึ้นมาทำการเพิ่มหรือลบกฎออกจากไอพีเทเบิล และทำการบันทึกการทำงานลงในแฟ้มข้อมูลชื่อ isagdfw.log เพื่อเก็บข้อมูลการทำงานเพื่อการตรวจสอบต่อไป